Technician Android — App Documentation

🎓 Intern Onboarding — Tirth Patel (Start: May 25, 2026)

Your assignment: Build an ML model (or hybrid statistical/ML approach) to validate well and lift station compliance data **as it is entered** into the Technician app — flagging anomalies before they reach the server.

What you need to understand from this document

- How the app authenticates technicians (phone number + SMS PIN)
- How reference data (routes, wells, lift stations, meters, readings) flows from the server to the device
- How readings are submitted (optimistic path → local pending fallback)
- The local Room database schema — this mirrors the server DB closely; it's where your model's input data will come from on-device
- The Service Line Inventory feature — secondary feature you should be aware of but don't need to focus on for the ML work

Your primary focus for the ML work is `ts_well_readings`, `ts_station_readings`, and the pending equivalents (`pending_well_readings`, `pending_station_readings`). See the Database documentation (`aquacertify-db-documentation.md`) for the server-side schema and historical data.

Table of Contents

Tech Stack

Component	Library / Version
Language	Kotlin
Min SDK	API 23 (Android 6.0)
Target / Compile SDK	API 36
App version	1.5.9 (versionCode 54)
UI	XML layouts + ViewBinding/DataBinding (not Compose)
Architecture	MVVM — Activities + ViewModels + LiveData
Networking	Retrofit 3 + OkHttp + Gson
Local database	Room (SQLite)
Background work	WorkManager
Maps	Google Maps SDK + Play Services Location
Navigation	AndroidX Navigation (SafeArgs)
HTTP base URL	`https://aquacertify.com/`

Project Structure

```
app/src/main/java/com/consolidatedutilities/liftstation/cloud/
│
├── activities/
│   ├── main/               MainActivity — splash / auth gate
│   ├── enterphone/         EnterPhoneActivity — phone number entry
│   ├── enterpin/           EnterPinActivity — SMS PIN entry
│   ├── routes/             RoutesActivity + RoutesViewModel — route list
│   ├── routestations/      RouteStationsActivity — stations in a route
│   ├── routewells/         RouteWellsActivity — wells in a route
│   ├── liftstation/        LiftStationActivity + LiftStationViewModel
│   ├── well/               WellActivity + WellViewModel
│   ├── map/                MapsActivity — GPS map of all entities
│   ├── gpswell/            GpsWellActivity — set GPS for a well
│   ├── gpsliftstation/     GpsLiftStationActivity — set GPS for a station
│   ├── changewellmeter/    ChangeWellMeterActivity — swap active meter
│   ├── changeliftstationdial/ ChangeLiftStationDialActivity
│   ├── liftstationadvanceauto/ LiftStationAdvanceAutoActivity (special case)
│   ├── settings/           SettingsActivity (disabled in nav menu)
│   └── servicelineinventory/
│       ├── home/           ServiceLineInventoryHome — route list for SLI
│       ├── neighborhoods/  SliNeighborhoodsActivity
│       ├── addresses/      SliAddressesActivity
│       ├── detail/         SliAddressDetailActivity + ViewModel
│       └── map/            SliMapActivity
│
├── api/
```

```
│   ├── Api.kt              Retrofit interface — all endpoints
│   ├── RetrofitClient.kt   Singleton Retrofit setup
│   └── ProgressResponseBody.kt  Download progress tracking
│
├── database/
│   ├── AppDatabase.kt      Legacy Room DB (pre-v3 sync)
│   ├── AppDAO.kt           Legacy DAO
│   └── techniciansync/
│       ├── TechnicianSyncDatabase.kt   Primary Room DB (v4)
│       ├── TechnicianSyncDao.kt        All queries
│       ├── TechnicianSyncEntities.kt   All ts_* entity classes
│       ├── PendingReadingEntities.kt   pending_* entity classes
│       ├── TechnicianSyncCache.kt      Maps API response → DB
│       └── TechnicianSyncQueryModels.kt Route/well/station query projections
│
├── models/
│   ├── technicianSync/     API request/response models for v3 sync
│   ├── liftstation/        LiftStation request/response models
│   ├── well/               Well request/response models
│   ├── serviceline/        ServiceLine request/response models
│   ├── routes/             Route model (used in query projections)
│   └── ...                 Other per-endpoint models
│
├── workers/
│   ├── TechnicianSyncWorker.kt     WorkManager: full background sync
│   └── PendingReadingsWorker.kt    WorkManager: flush offline readings
│
├── TechnicianSyncRepository.kt     Singleton: orchestrates v3 sync calls
├── AppSharedPreferences.kt         JWT, userId, sync timestamp storage
├── MyApplication.kt                Application class
└── Utils.kt                        Date formatting + helpers
```

Authentication Flow

```
[Splash — MainActivity]
        │
        ▼ (no JWT in SharedPreferences)
[EnterPhoneActivity]
  - Technician types their phone number
  - Google Identity SMS hint auto-fills phone if available
  - POST /get-pin → server sends SMS OTP to that number
        │
        ▼
[EnterPinActivity]
  - Technician types 4-digit PIN from SMS
  - SmsRetriever API can auto-read the SMS if hash matches
  - POST /login → server returns JWT + userId
  - JWT + userId saved to SharedPreferences
        │
        ▼ (JWT now exists)
[RoutesActivity]
```

SharedPreferences keys (file: `lscloud`):

Key	Purpose
`jwt`	Bearer token for all API calls
`userId`	Technician's user ID (Int as String)
`technician_sync_since`	Unix epoch millis — incremental sync cursor
`force_offline`	Boolean — suppresses API calls after server errors
`offlineTimestamp`	Millis — when `force_offline` expires (2.5 hrs)

Please note `offlineTimestamp` is from a poor design choice I made to force the app in a limited offline mode for however long. Most of the dead code has been removed, but this is not longer functional. The app caches everything it needs and runs smoothly if the server goes down.

JWT format: Passed as a raw `Authorization` header value (the server expects the format it issued — typically `Bearer <token>`). All `Api` interface methods accept `authToken: String` and pass it directly.

Data Sync Architecture

The v3 sync is the heart of the app. On login and periodically, the app downloads its full dataset from one endpoint.

The Sync Endpoint

```
POST /api/v3/sync-data/technician
Header: Authorization: <jwt>
Body: { "since": <unix_epoch_millis or null> }
```

- `since = null` on first sync — server returns full dataset (all routes, wells, stations, readings, etc.)
- `since = <timestamp>` on subsequent syncs — server returns only records modified after that timestamp
- The response includes a `since` field — the app saves this as the next incremental cursor

Note: The current code sends `since = 0L` (not null) on first sync. This is a known issue — `0L` means epoch (Jan 1 1970), which the server interprets as "give me everything since 1970" rather than "give me everything." It works in practice but is semantically incorrect. The correct behavior would be to send `null`. See Key Known Issues.

TechnicianSyncRepository

```
// Singleton — get with:
TechnicianSyncRepository.getInstance(context)

// Call to trigger sync:
repository.sync(
    onComplete = { /* called when done or failed */ },
    onProgress = { progress -> /* 0–100 download progress */ }
)
```

`sync()` does:

1. Reads JWT and `since` from SharedPreferences
2. Calls `POST /api/v3/sync-data/technician`
3. On success: saves new `since` to SharedPreferences, calls `TechnicianSyncCache.cacheSyncResponse(body)` on a background coroutine
4. `TechnicianSyncCache` upserts all entities into the Room database in one transaction-like sequence

When Sync Runs

Trigger	What happens
App opens fresh (RoutesActivity, routes list empty)	Full progress-bar sync
Pull-to-refresh on RoutesActivity	Triggers sync + refreshes route list
WorkManager — `TechnicianSyncWorker`	Background sync on schedule
User returns to RoutesActivity (onResume, routes already loaded)	No re-sync — cached data stays

Local Room Database

The primary database is `lscloud_technician_sync` (Room, version 4). All data from the v3 sync is stored here. This is what the app reads for display — it never reads from the network for individual screens after sync completes.

Entity Map

Reference / Metadata Tables

Room Entity	Table name	Description
`TsSyncMetaEntity`	`ts_sync_meta`	Single row — stores `since` timestamp a `cachedAtEpochSec` (shown in "Last Sync UI)
`TsStateEntity`	`ts_states`	US state abbreviatio
`TsAquiferEntity`	`ts_aquifers`	Georgia aquifer nam (31 rows)
`TsCountyEntity`	`ts_counties`	GA counties with FIPS/GWW codes
`TsPermissionsEntity`	`ts_permissions`	Single row — this technician's feature f

Room Entity	Table name	Description
`TsOrganizationEntity`	`ts_organizations`	The org the technicia belongs to
`TsSystemEntity`	`ts_systems`	Water systems (publ water supplies)
`TsSafeDrinkingWaterPermitEntity`	`ts_safe_drinking_water_permits`	SDWA permit record
`TsGroundwaterWithdrawalPermitEntity`	`ts_groundwater_withdrawal_permits`	GWW permits for we

Operational Data Tables

Room Entity	Table name	Description
`TsUserEntity`	`ts_users`	All technicians with access to this org
`TsRouteEntity`	`ts_routes`	Route names and IDs
`TsWellEntity`	`ts_wells`	Well metadata (name, lat/long, route, system, permits)
`TsLiftStationEntity`	`ts_lift_stations`	Lift station metadata
`TsWellMeterEntity`	`ts_well_meters`	Meters installed on wells (active flag, moving digits, fixed zeros)
`TsStationDialEntity`	`ts_station_dials`	Dials on lift stations (pump 1 / pump 2, decimal places)
`TsWellReadingEntity`	`ts_well_readings`	Historical well readings synced from server
`TsStationReadingEntity`	`ts_station_readings`	Historical station readings synced from server

SLI (Service Line Inventory) Tables

Room Entity	Table name	Description
`TsAddressEntity`	`ts_addresses`	Service addresses (meter connections)
`TsNeighborhoodEntity`	`ts_neighborhoods`	Named neighborhoods per route
`TsAddressMetaEntity`	`ts_address_meta`	Property type / land use metadata
`TsServiceLineEntity`	`ts_service_lines`	Service line classification records
`TsServiceLineResponseEntity`	`ts_service_line_responses`	Lookup values (ownership types, materials, basis options, etc.) — composite PK on `(type, id)`

Pending / Offline Tables

Room Entity	Table name	Description
`PendingWellReadingEntity`	`pending_well_readings`	Well readings saved locally, not yet confirmed by server
`PendingStationReadingEntity`	`pending_station_readings`	Station readings saved locally, not yet confirmed
`PendingServiceLineEntity`	`pending_service_lines`	SLI classifications saved locally, not yet confirmed

Key Entity Field Details

`TsWellMeterEntity` **(ts_well_meters)**

Field	Type	Notes
`id`	Int PK	
`wellId`	Int	FK → ts_wells
`movingDigits`	Int	How many digits can change (the rolling display)
`fixedZeros`	Int	Trailing zeros always present (multiplier: 10^fixedZeros)
`active`	Boolean	Only one meter should be active per well at a time
`installationDate`	String	ISO date
`intialReading`	Double	Typo is intentional — matches server field name (missing 'i')
`createdBy` / `modifiedBy`	Int	User IDs

How meter readings work: The display input is constrained to `movingDigits` characters (e.g., 9 digits). The actual raw integer stored in `ts_well_readings.meterReading` is what the technician types. To get daily volume pumped: `today.meterReading - yesterday.meterReading` (accounting for rollover when today < yesterday).

`TsStationDialEntity` **(ts_station_dials)**

Field	Type	Notes
`dialId`	Int PK	
`liftStationId`	Int	FK → ts_lift_stations
`pumpNumber`	Int	1 or 2
`digitsLeftOfDecimal`	Int	Integer digits in the display (typically 5)
`digitsRightOfDecimal`	Int	Decimal digits (typically 1)

Field	Type	Notes
`active`	Boolean	
`initialReading`	Double	Reading at installation

`TsWellReadingEntity` **(ts_well_readings)**

Field	Type	Notes
`id`	Int PK	
`wellId`	Int	
`meterReading`	Double	Raw meter value from dial
`chlorineResidual`	Double?	mg/L — nullable
`dateOfReading`	String	"YYYY-MM-DD"
`userId`	Int	Who submitted
`comment`	String?	Optional comment
`meterId`	Int	Which meter was active at time of reading
`created / modified`	Long	Epoch millis

`TsStationReadingEntity` **(ts_station_readings)**

Field	Type	Notes
`id`	Int PK	
`liftStationId`	Int	
`dial1Id`	Int	Pump 1 dial used
`dial2Id`	Int	Pump 2 dial used
`pump1`	Double	Cumulative pump 1 reading
`pump2`	Double	Cumulative pump 2 reading
`rainfall`	Double?	Inches — nullable
`dateOfReading`	String	"YYYY-MM-DD"
`userId`	Int	
`comment`	String?	

`TsPermissionsEntity` **(ts_permissions)**

One row. Controls which nav menu items are visible for this technician:

Field	What it unlocks
`changeWellMeter`	"Change Well Meter" menu item
`changeLiftStationDial`	"Change Lift Station Dial" menu item
`setGPSLocationWell`	"Add Location — Well" menu item
`setGPSLocationLiftStation`	"Add Location — Station" menu item
`accessServiceLineInventory`	"Service Line Inventory" menu item

Pending Readings — Offline Support

If a reading can't be sent to the server immediately (network error, offline), it's saved to a local pending table. The UI reflects this with a **light blue (#cde1ff)** button color instead of green.

Write Path (WellViewModel / LiftStationViewModel)

```
Technician submits reading
        │
        ▼
Try optimistic send: POST /wells/{id}/reading/{date}
        │                                   │
   Success (200)                       Failure / offline
        │                                   │
Delete from pending_*            Save to pending_well_readings
Mark button green                       or pending_station_readings
                                  Schedule PendingReadingsWorker
                                  Mark button light blue
```

The WorkManager job uses:

```
Constraints.Builder()
    .setRequiredNetworkType(NetworkType.CONNECTED)
    .build()
```

So the pending flush only runs when the device is connected.

PendingReadingsWorker

Runs when:

- Explicitly scheduled via `WorkManager.enqueueUniqueWork("flush_pending_readings", KEEP, request)` after a failed submission
- Also handles `pending_service_lines` for SLI classifications

For each pending record it:

1. Tries the API call (`.execute()` — synchronous, safe in a Worker)
2. On HTTP 200 OR HTTP 409 (already exists — duplicate): deletes from pending table
3. On any other failure: sets `allSucceeded = false` → returns `Result.retry()`

HTTP 409 handling is intentional: If the server already has a reading for that entity/date, the pending record is considered resolved and removed.

Screen-by-Screen Walkthrough

MainActivity

Entry point. Checks SharedPreferences for JWT:

- JWT present → launch RoutesActivity, finish self
- No JWT → show splash screen with a "Get Started" button → EnterPhoneActivity

EnterPhoneActivity

- Phone number input with `PhoneNumberFormattingTextWatcher`
- Google Identity Phone Hint popup can auto-fill the number
- `POST /get-pin` → server sends OTP SMS
- Navigates to EnterPinActivity

EnterPinActivity

- 4-digit PIN input
- `SmsRetriever` API watches for incoming SMS and auto-fills if the SMS hash matches the app's signing key
- `POST /login` → gets JWT + userId
- Saves both to SharedPreferences
- Launches RoutesActivity

RoutesActivity

The main hub. Shows:

- Technician name (from `ts_users` via `ts_permissions`)
- Organization name (from `ts_organizations`)
- "Last Synced: M/d/yy h:mm AM/PM" (from `ts_sync_meta.cachedAtEpochSeconds`)
- Route list with reading progress (see Button Color Logic)
- Side drawer nav menu (conditionally shows items based on permissions)

On first load: Shows progress bar during full sync. On subsequent `onResume` calls (returning from a sub-screen), does NOT re-sync — routes are already loaded from Room.

Pull to refresh: Re-fetches route data from Room and triggers a new incremental sync.

RouteStationsActivity / RouteWellsActivity

Lists all lift stations or wells on a route. Each item shows a colored button:

- Red: no reading today
- Green: reading confirmed
- Light blue (#cde1ff): reading saved locally (pending)

Tap an item → LiftStationActivity or WellActivity.

WellActivity

The core reading entry screen for a well. Shows:

- Well name
- Previous reading (date, meter value, chlorine residual, technician, comment)
- "Reading unavailable" message if no previous reading exists
- Today's reading fields (or pre-filled if already submitted)
- Meter input constrained by the active meter's `movingDigits + fixedZeros`
- Chlorine residual input (format: `X.X`, validated with regex)
- Comment field
- Submit button

Validation:

- Meter reading must match `^\\d{1,<movingDigits>}<fixed_zeros>$`
- Residual must match `^[0-9]\\.[0-9]$`
- If today's reading is lower than yesterday's, a dialog warns the technician (meter rollover scenario — they can proceed or cancel)
- If this is the first reading for this meter, the lower-reading dialog is skipped

Submit flow:

1. Tries `POST /wells/{id}/reading/{date}`
2. If network fails → saves to `pending_well_readings` → schedules `PendingReadingsWorker`

LiftStationActivity

Same pattern as WellActivity, but for lift stations:

- Two pump inputs (pump 1 + pump 2)
- Rainfall input
- Formatted to `digitsRightOfDecimal` decimal places based on the active dial
- Validates both pumps before allowing submit

LiftStationAdvanceAutoActivity

Special-cased activity for the "Advance Auto" lift station — handles a different pump configuration or display quirk specific to that station.

MapsActivity

Displays all wells and lift stations on a Google Map. Can filter by route. Tapping a pin opens the detail screen for that entity.

GpsWellActivity / GpsLiftStationActivity

Lets authorized technicians update the GPS coordinates for a well or lift station. Uses Play Services fused location provider to get current device location, then `POST /wells/{id}/gps` or `/lift-stations/{id}/gps`.

ChangeWellMeterActivity / ChangeLiftStationDialActivity

Allows authorized technicians to swap out the active meter/dial on an entity (e.g., after a physical meter replacement). Permission-gated via `ts_permissions`.

API Layer

All endpoints are defined in `Api.kt` (Retrofit interface). The base URL is `https://aquacertify.com/`.

Auth Endpoints (no JWT required)

Method	Endpoint	Purpose
`POST`	`/get-pin`	Trigger SMS OTP
`POST`	`/login`	Exchange phone + PIN for JWT

V3 Sync (JWT required)

Method	Endpoint	Purpose
`POST`	`/api/v3/sync-data/technician`	Incremental data sync — main sync endpoint
`POST`	`/api/v3/service-lines`	Add SLI classification
`PUT`	`/api/v3/service-lines`	Update SLI classification

Legacy Endpoints (JWT required)

These are older endpoints that predate the v3 sync architecture. They're still called for reading submission and a few management actions.

Method	Endpoint	Purpose
GET	/routes/users/{id}/{date}	Get routes for user (legacy)
GET	/routes/{id}/items/{date}	Get wells + stations on route (legacy)
GET	/wells/{id}/reading/{date}	Get a well reading for date
POST	/wells/{id}/reading/{date}	Submit a well reading
PUT	/wells/{id}/reading/{date}	Update a well reading
GET	/lift-stations/{id}/reading/{date}	Get a station reading
POST	/lift-stations/{id}/reading/{date}	Submit a station reading
PUT	/lift-stations/{id}/reading/{date}	Update a station reading
POST	/wells/{id}/meter	Change active meter on a well
POST	/lift-stations/{id}/dial/{dialNumber}	Change active dial
POST	/wells/{id}/gps	Set GPS coordinates for a well
POST	/lift-stations/{id}/gps	Set GPS for a lift station
POST	/sync-data/insert-readings	Batch reading sync (legacy)
POST	/sync-data/well-meter	Sync meter changes (legacy)
GET	/sync-data/get-data	Full data sync (legacy, pre-v3)

Custom Headers (added by OkHttp interceptor)

Every request automatically includes:

```
acertify-os: Android <version>
acertify-technician-version: <app_versionName>
acertify-phone-make-model: <device_model>
```

Request / Response Models

Sync request:

```
data class TechnicianSyncRequest(val since: Long)
```

Well reading submission:

```
data class WellInsertRequest(
    val meterReading: String,
    val residual: String,    // "0.0" if not provided
```

```
    val comment: String?
)
```

Station reading submission:

```
data class LiftStationRequest(
    val pump1: String,
    val pump2: String,
    val rainfall: String,
    val comment: String?
)
```

Service Line Inventory

The SLI module is a secondary feature in the app, accessible via the nav drawer to technicians with `accessServiceLineInventory = true`.

Purpose: Field technicians can look up addresses on their routes and classify the service line material (lead / non-lead / unknown) as part of the EPA/EPD Lead and Copper Rule compliance program.

SLI Navigation

```
ServiceLineInventoryHome (route list)
        │
        ▼
SliNeighborhoodsActivity (neighborhoods in route)
        │
        ▼
SliAddressesActivity (addresses in neighborhood)
        │
        ▼
SliAddressDetailActivity (classify a specific address)
```

There is also `SliMapActivity` — shows addresses on a map.

SLI Classification Flow (SliAddressDetailViewModel)

1. Loads address from `ts_addresses` by addressId
2. Loads existing classification from `ts_service_lines` (if any)
3. Loads all lookup options from `ts_service_line_responses` by type:
    - `"ownership_type"` — Split / System / Customer / Unknown
    - `"system_owned_material"` — Non-Lead / GNRR / GRR / Lead / Unknown / N/A
    - `"customer_owned_material"` — same options
    - `"basis_of_classification"` — Historical Documentation, Excavation, Visual Inspection, etc.
    - `"installation_date"` — Pre 1990 / Post 1990 / Unknown
    - `"presence_of_lead_connector"` — Yes / No / Unknown

- `"building_type"` — SFR / MFR / Non-Residential / w/ Childcare
- `"school_or_childcare_facility"` — Yes (types) / No

4. Technician fills out the form
5. Submit tries `POST /api/v3/service-lines` or `PUT` if updating
6. On failure → saves to `pending_service_lines` → schedules `PendingReadingsWorker`

Button Color Logic

Route and entity button colors are derived by the DAO query `observeRouteProgressForDate(date)`:

State	Color	Meaning
No readings (server OR pending)	Red	Not yet visited today
All readings present	Green	Fully complete for today
Some readings present	Yellow	Partially complete
Any pending (offline) readings	Light blue (#cde1ff)	Saved locally, not yet confirmed

The query counts records from both `ts_well_readings` / `ts_station_readings` (confirmed) and `pending_well_readings` / `pending_station_readings` (pending) for today's date, joined with `ts_wells` and `ts_lift_stations` by `routeId`.

Key Known Issues

These are carried forward from prior code review — Brian is aware of them:

1. `since` **defaults to** `0L` **not** `null` **on first sync.** In `TechnicianSyncRepository`, `sinceForRequest` defaults to `0L` when no sync has been done. The server should receive `null` to trigger the full initial dataset. Currently `0L` (epoch) is sent, which works because the server returns everything since the epoch, but is semantically wrong.
2. `intialReading` **typo.** The field `intialReading` (missing 'i') exists in `SyncWellMeter`, `TsWellMeterEntity`, and `TechnicianSyncModels.kt`. This matches the server API field name, so fixing it would require a coordinated change.
3. **Nullable fields declared non-null in entities.** Several entity fields (e.g., in `TsWellMeterEntity`) could crash the app if the API returns `null` for a field typed as non-nullable in Kotlin. This is low-risk in practice but worth knowing.
4. **Bare** `CoroutineScope(Dispatchers.IO)` **in** `TechnicianSyncRepository`**.** Not lifecycle-safe — could leak if the repository is destroyed. `viewModelScope` or `applicationScope` would be more appropriate.

Your ML Integration Point

When a technician submits a reading in `WellViewModel` or `LiftStationViewModel`, there is a clear intercept point **before the API call** where you can inject a validation check.

Suggested integration in WellViewModel

```
// Where the submit button is handled, before calling the API:
fun submitReading(wellId: Int, date: String, meterReading: String, residual: String, comment: String)

    // TODO (Tirth): Load historical readings from ts_well_readings for this wellId
    // Run your model — is this reading plausible given the history?
    // val anomalyScore = mlModel.score(wellId, meterReading.toDouble(), previousReadings)
    // if (anomalyScore > THRESHOLD) { showWarningDialog() }

    // Then proceed to API call / pending fallback
}
```

What data is available on-device at submission time

Because the full sync has already run, the local Room database has everything you need:

```
// Get all historical readings for a well (in order)
technicianSyncDao.getWellReadingsForWell(wellId)  // List<TsWellReadingEntity>

// Get the active meter (for scale context)
technicianSyncDao.getActiveWellMeter(wellId)       // TsWellMeterEntity?

// For stations:
technicianSyncDao.getStationReadingsForStation(liftStationId)  // List<TsStationReadingEntity>
```

Model deployment options

Option	Pros	Cons
On-device (TFLite / Core ML)	Works offline, zero latency	Model size, update distribution
Server-side inference endpoint	Easy to update, no size constraint	Requires network (unavailable when offline)
Hybrid: on-device for immediate feedback, server for logging	Best UX	More complex

Brian's recommendation: start with a simple statistical approach (z-score on daily deltas per well/station) before committing to a full TFLite deployment. The on-device historical data is sufficient to compute per-entity baselines without a trained model.

Anomalies to flag

Well readings:

- `meterReading < previousReading` — possible meter rollover or typo (app already warns, but not validated)
- `(meterReading - previousReading)` is more than 3σ above the historical daily delta for this well
- `chlorineResidual < 0.2` mg/L — below regulatory minimum
- `chlorineResidual > 4.0` mg/L — above EPA maximum residual disinfectant level

Lift station readings:

- `pump1 < previousPump1` or `pump2 < previousPump2` — possible reset/rollover
- Daily pump delta is an extreme outlier for this station (>3σ)
- `pump1` and `pump2` deltas are wildly divergent from their historical ratio
- `rainfall < 0` or `rainfall > 20` (inches in one day)

DB Connection for Historical EDA

See `aquacertify-db-documentation.md` for the server-side schema, sample data, and the SQL queries to export all well and station readings for exploratory data analysis.

The on-device Room database mirrors the server schema closely — column names are camelCase in Kotlin but map directly to the snake_case server columns.

Repo: `git@github.com:consoli-tech/Technician-Android.git`
Branch for your work: `technician-ml`
App version as of this writing: 1.5.9 (versionCode 54)
Package: `com.consolidatedutilities.liftstation.cloud`